

// redes de computadores II • aula 01

Regras e endereços

Fundamentos para investigar redes com Python.

```
// aula01/laboratorio_rede.py
protocolo = "http"
servidor = "127.0.0.1"
porta = 8000
recurso = "/"
```

// método

Nesta disciplina, explicações serão confrontadas com testes

01

Executar aplicações

Código rodando, não apenas descrito.

02

Observar tráfego

Ver o que sai e o que chega pela rede.

03

Alterar condições

Mudar uma variável por vez e comparar.

04

Explicar resultados

Dizer por que aconteceu, com os termos certos.

// abertura

02 / 33

// objetivos verificáveis

Você vai identificar e representar uma comunicação

- [] Identificar quem inicia uma solicitação e quem a atende.
- [] Explicar o papel do endereço IP e da porta.
- [] Reconhecer HTTP como protocolo de aplicação no exemplo usado.
- [] Diferenciar, de forma introdutória, TCP e UDP.
- [] Executar e modificar um script que representa o endereço de um serviço.

// pergunta inicial

Um endereço escrito não garante um serviço disponível

“Se eu escrever `http://127.0.0.1:8000/`, já existe um site funcionando?”

TEXTO

Uma URL que alguém digitou

É uma sequência de caracteres. Descreve para onde um pedido poderia ser enviado e qual protocolo usar. Existe mesmo que nada esteja rodando.

`http://127.0.0.1:8000/`

SERVIÇO

Um processo executando e atendendo

É um programa em execução, associado àquele endereço e àquela porta, pronto para receber solicitações e responder segundo as regras do protocolo.

`processo escutando em :8000`

// pergunta inicial

A URL reúne instruções para localizar e solicitar um recurso

`http://127.0.0.1:8000/index.html`

Uma URL que alguém digitou

`http` → protocolo de aplicação utilizado;

`127.0.0.1` → endereço IP do destino;

`8000` → porta do serviço;

`/index.html` → caminho do recurso solicitado.

A URL descreve uma tentativa de acesso; ela não garante que exista um serviço funcionando.

// pergunta inicial

A URL reúne instruções para localizar e solicitar um recurso

`http://127.0.0.1:8000/index.html`

Uma URL que alguém digitou

`http` → protocolo de aplicação utilizado;

`127.0.0.1` → endereço IP do destino;

`8000` → porta do serviço;

`/index.html` → caminho do recurso solicitado.

A URL descreve uma tentativa de acesso; ela não garante que exista um serviço funcionando.

O endereço IP identifica uma interface na rede"

- ❑ O IP participa da identificação do destino da comunicação.
- ❑ `127.0.0.1` representa o próprio computador por meio da interface de loopback.
- ❑ O endereço pode mudar de acordo com a rede e sua configuração.
- ❑ Digitar um IP não garante que exista um serviço disponível.

O IP funciona como o endereço do prédio: ajuda a localizar o destino.

// pergunta inicial

A URL reúne instruções para localizar e solicitar um recurso

`http://127.0.0.1:8000/index.html`

Uma URL que alguém digitou

`http` → protocolo de aplicação utilizado;
`127.0.0.1` → endereço IP do destino;
`8000` → porta do serviço;
`/index.html` → caminho do recurso solicitado.

A URL descreve uma tentativa de acesso; ela não garante que exista um serviço funcionando.

A porta direciona a comunicação ao processo correto

- ❑ A **porta** é um identificador numérico usado pelos protocolos de transporte.
- ❑ Um computador pode executar vários serviços simultaneamente.
- ❑ O servidor do laboratório usará a porta 8000.
- ❑ A porta precisa estar associada a um processo que esteja aguardando comunicações.

Se o IP indica o prédio, a porta ajuda a encontrar a sala de atendimento.

O endereço IP identifica uma interface na rede

192.168.0.15

Quatro números em IPv4. Esse endereço identifica uma interface de rede de um host dentro da rede em que ele está.

O endereço leva a mensagem até a máquina certa — e para aí.

LIMITE DA ANALOGIA POSTAL

A casa tem um endereço fixo. Um host pode:

- ter mais de uma interface (Wi-Fi e cabo);
- receber endereço diferente ao reconectar;
- ter um endereço só válido dentro da rede local.

// conceito · porta

A porta distingue pontos de comunicação no host

127.0.0.1:8000

Porta do serviço que vamos executar no laboratório da próxima aula.

127.0.0.1:9000

Mesmo host, outro ponto de comunicação: outro serviço — ou nenhum.

A porta só identifica um ponto de comunicação em conjunto com o endereço e o protocolo de transporte. Trocar a porta muda o destino da tentativa; não cria nem move serviço nenhum.

8000 padrão adotado pela comunidade para testes locais — não é a porta padrão do HTTP.

A URL reúne informações para acessar um recurso

http://127.0.0.1:8000/

http

Protocolo

Regras da troca na camada de aplicação.

127.0.0.1

Endereço

Para qual interface enviar o pedido.

8000

Porta

Qual ponto de comunicação no host.

/

Caminho

Qual recurso está sendo pedido (raiz).

Uma URL também pode trazer um nome em lugar do número. Como o nome é traduzido em endereço é assunto de aula posterior.

// conceito • loopback

Loopback mantém a comunicação no próprio computador

127.0.0.1

Endereço IPv4 de loopback: aponta para o próprio host. A mensagem sai de um processo e volta para outro processo da mesma máquina.

Uma aplicação local usa a pilha de rede do sistema sem depender de acesso à internet.

POR QUE USAR NO LABORATÓRIO

Funciona sem internet e sem configuração de rede.

Isola o experimento: o erro está no nosso código, não no cabo.

Cada aluno testa no próprio computador, sem interferir nos colegas.

// conceito • loopback

Cliente e servidor precisam usar regras compatíveis

Considere um servidor esperando requisições HTTP na porta **8000**.

O cliente envia:

```
GET /index.html HTTP/1.1  
Host: 127.0.0.1:8000
```

O servidor reconhece a estrutura HTTP e consegue interpretar:

“O cliente está pedindo o recurso /index.html.”

Agora imagine que o cliente envie apenas:

“OI SERVIDOR ME MANDA A PAGINA”

Os dados podem até chegar ao processo servidor, mas ele espera uma mensagem HTTP. Como o formato recebido não segue as regras esperadas, o servidor pode:

- Rejeitar a mensagem;
- Retornar uma resposta de erro;
- Encerrar a conexão;
- Não conseguir processar corretamente o pedido.

// bloco B · exposição · 01

Os programas assumem papéis na comunicação

Cliente, servidor e protocolo: três ideias que descrevem qualquer troca entre aplicações.

O cliente inicia a solicitação ao serviço

Cliente é um papel, não um equipamento

É o processo que decide começar a conversa: monta uma solicitação, envia e espera a resposta.

No nosso exemplo, o navegador é o cliente: você informa o endereço e ele faz o pedido.

O CLIENTE

- escolhe o momento do pedido;
- precisa saber endereço, porta e protocolo;
- não precisa estar sempre em execução.

O mesmo computador pode ter vários processos clientes ao mesmo tempo.

O servidor aguarda e atende solicitações

Processo servidor

Um programa em execução que fica associado a um endereço e a uma porta, esperando pedidos.

Ele só atende enquanto está rodando. Fechou o programa, o serviço deixa de responder.

Computador servidor

É o equipamento onde o processo roda. Pode ser um servidor de datacenter — ou o PC do laboratório.

// diagrama • troca de mensagens

Forma de comunicação

CLIENTE

Navegador

Processo que inicia o pedido

solicitação →

mensagem de aplicação (HTTP)

← resposta

conteúdo + status

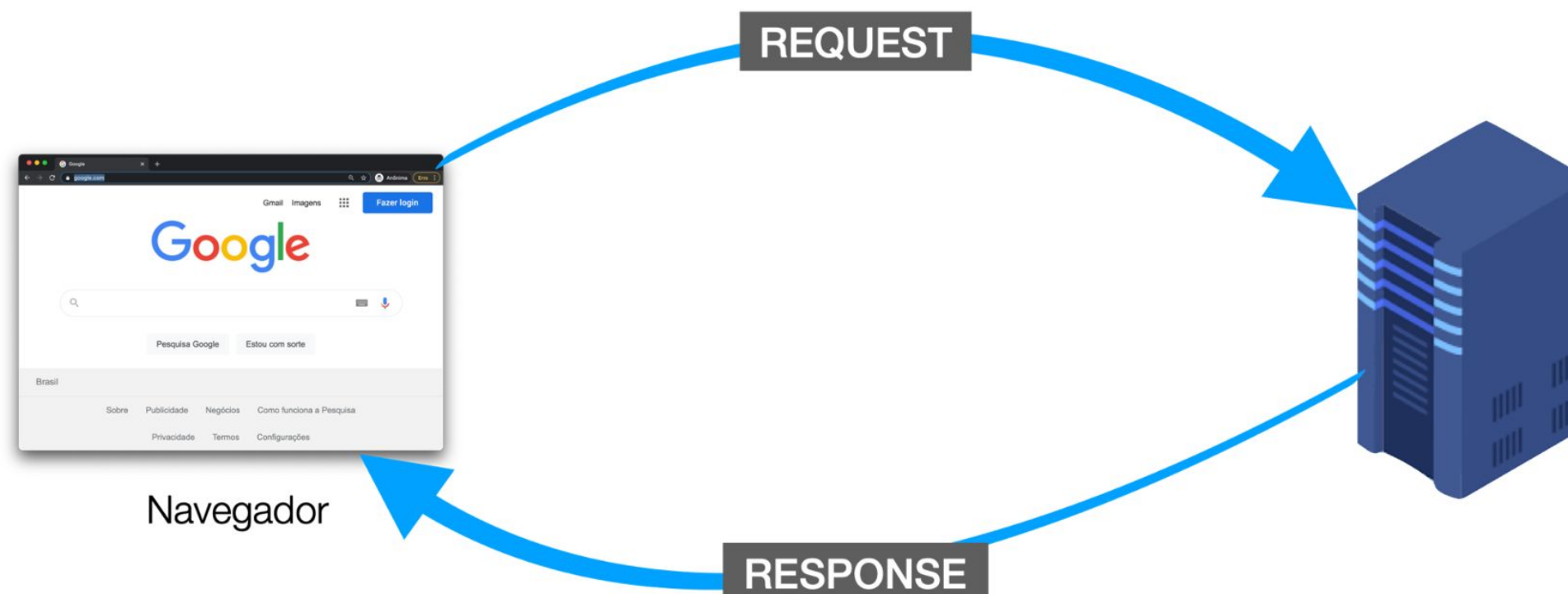
SERVIDOR

Processo servidor

Em execução, escutando na porta

CLIENTE

SERVIDOR



// exposição

08 / 33

// comparação · onde ficam os papéis

Cliente e servidor podem estar no mesmo computador

ENTRE EQUIPAMENTOS

Dois computadores diferentes

A mensagem sai da máquina, atravessa a rede e chega à interface do outro host.

COMUNICAÇÃO LOCAL

O mesmo computador nos dois papéis

Dois processos na mesma máquina conversam usando a pilha de rede, sem sair do host.

O protocolo define como interpretar a troca

Um protocolo estabelece regras para trocar e interpretar informações: o que se pode pedir, em que formato e como a resposta será entendida.

HTTP define regras de comunicação de aplicação — é o protocolo do nosso exemplo.

EXEMPLO DE REGRA COMBINADA

```
cliente: quero o recurso /  
servidor: aqui está o conteúdo de /
```

Os dois lados sabem quem fala primeiro, o que cada mensagem significa e o que fazer quando o pedido não pode ser atendido.

// comparação · onde ficam os papéis

Protocolo

DEFINIÇÃO

Um protocolo de rede é um conjunto de regras que define o formato, o significado e a sequência das mensagens trocadas entre entidades de comunicação, além das ações esperadas durante essa troca.

// comparação · onde ficam os papéis

Protocolo

Formato da mensagem — sintaxe

O protocolo define como a mensagem deve ser escrita e organizada. O protocolo define como a mensagem deve ser escrita e organizada. O protocolo define como a mensagem deve ser escrita e organizada.

Por exemplo, uma requisição HTTP pode começar assim:

```
1 GET /index.html HTTP/1.1
2 Host: 127.0.0.1:8000
3
4
5
```

- ❑ **GET** é o método;
- ❑ **/index.html** é o recurso solicitado;
- ❑ **HTTP/1.1** indica a versão do protocolo;
- ❑ **Host** é um cabeçalho;
- ❑ Uma linha vazia indica o fim dos cabeçalhos.

// comparação · onde ficam os papéis

Protocolo

Significado da mensagem — semântica

<https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status>

Além do formato, o protocolo define o significado de cada elemento.

No HTTP:

- 1 ☐ **GET** significa solicitar uma representação de um recurso;
- 2 ☐ **POST** normalmente envia dados para processamento;
- 3 ☐ **200** OK indica que a solicitação foi atendida;
- 4 ☐ **404** Not Found indica que o recurso solicitado não foi encontrado;
- 5 ☐ **500** Internal Server Error indica uma falha no servidor ao processar a solicitação.

// comparação · onde ficam os papéis

Protocolo

Ordem da comunicação — sequência

O protocolo também estabelece a ordem esperada dos eventos.

O servidor não envia uma resposta para uma requisição que ainda não recebeu. O cliente, por sua vez, precisa saber que determinada mensagem é uma resposta à sua solicitação.

No caso mais simples do HTTP:

- 1 ☐ Cliente envia uma requisição
- ↓
- 2 ☐ Servidor interpreta e processa
- ↓
- 3 ☐ Servidor envia uma resposta
- ↓
- 4 ☐ Cliente interpreta a resposta

sintaxe → semântica → sequência →

// comparação · onde ficam os papéis

Protocolo

Comportamento diante dos eventos

O protocolo também define ou orienta o que deve acontecer em determinadas situações.

Por isso, um protocolo não é apenas “o idioma” da comunicação. Ele também contém regras de organização e comportamento.

No caso mais simples do HTTP:

- 1 ☐ O que fazer quando a mensagem é válida;
- 2 ☐ Como informar que o recurso não existe;
- 3 ☐ Como indicar erro de autenticação;
- 4 ☐ Como sinalizar que a solicitação está malformada;
- 5 ☐ Quando manter ou encerrar a comunicação;
- 5 ☐ Como identificar a versão do protocolo utilizada.

// comparação · onde ficam os papéis

O HTTP define a conversa das aplicações web

HTTP significa **Hypertext Transfer Protocol**. É um protocolo da camada de aplicação usado para organizar solicitações e respostas entre aplicações.

Em uma comunicação web:

- O navegador geralmente atua como cliente HTTP;
- O servidor web atua como servidor HTTP;
- O cliente envia uma requisição;
- O servidor devolve uma resposta.

Exemplo de requisição

```
1 GET /index.html HTTP/1.1
2 Host: 127.0.0.1:8000
3 Accept: text/html
```

Exemplo de resposta

```
1 HTTP/1.1 200 OK
2 Content-Type: text/html
3 Content-Length: 125
```

// comparação · onde ficam os papéis

Onde entra o HTTPS?

HTTPS não é um protocolo totalmente separado do HTTP no sentido de substituir suas operações.

“HTTPS é a comunicação HTTP protegida por TLS.”

O HTTP continua definindo:

- Método;
- Caminho;
- Cabeçalhos;
- Corpo;
- Código da resposta.

0 TLS acrescenta:

- Confidencialidade;
- Integridade;
- Autenticação do servidor por certificado.

// comparação · onde ficam os papéis

O TCP oferece um serviço de transporte

O HTTP define o conteúdo e o significado da conversa da aplicação. Porém, essa mensagem precisa ser transportada entre os processos.

É aí que entra o TCP.

TCP oferece às aplicações um fluxo de bytes confiável e ordenado entre dois pontos da comunicação.

- Entrega dos bytes;
- Entrega na ordem correta;
- Detecção e recuperação de perdas;
- Controle para evitar que o receptor seja sobrecarregado;
- Identificação dos processos por portas;
- Estabelecimento de uma conexão antes da troca de dados.

Exemplo de requisição

```
1 Cliente: 127.0.0.1:52341
2 Servidor: 127.0.0.1:8000
3
```

O UDP oferece outro tipo de transporte

UDP também é um protocolo de transporte, mas possui características diferentes.

UDP permite enviar datagramas sem estabelecer uma conexão como a do TCP e sem oferecer, por si só, garantia de entrega, ordem ou retransmissão.

- Mantém os limites dos datagramas;
- Possui menos mecanismos do que o TCP;
- Não confirma a entrega por conta própria;
- Não reorganiza datagramas fora de ordem;
- Não retransmite automaticamente dados perdidos;
- Também utiliza portas para identificar os pontos da comunicação.

Características de uso do UDP

- 1 Tolerar determinadas perdas;
 - 2 Priorizar baixa sobrecarga;
 - 3 Implementar seus próprios mecanismos de controle;
- Realiza consultas curtas;
Utiliza protocolos construídos especificamente sobre UDP.

TCP e UDP oferecem serviços de transporte diferentes

CRITÉRIO	TCP	UDP
Organização	Fluxo contínuo de bytes	Datagramas independentes
Conexão	Estabelece conexão antes de enviar dados	Envia sem estabelecer conexão
Ordem	Entrega os dados na ordem enviada	Não reordena o que chega fora de ordem
Entrega	Retransmite o que falta, enquanto a comunicação for viável	Não oferece, por si só, garantia de entrega

Não é “TCP lento × UDP rápido”: são serviços diferentes. TCP não garante sucesso em qualquer situação, e UDP não significa que tudo se perde.

// diagrama • cenário do laboratório (HTTP/1.x)

A aplicação utiliza serviços das camadas inferiores

CLIENTE

Navegador

SERVIDOR

Servidor HTTP em Python

HTTP/1.x

regras da troca de aplicação

TCP

fluxo de bytes ordenado e confiável

IP

endereçamento — 127.0.0.1

Caminho local de loopback

a mensagem não sai do computador

Cenário HTTP/1.x deste laboratório. Outras versões de HTTP podem usar transporte diferente.

Papéis, endereços e protocolos têm funções distintas

"Servidor precisa ser outro computador."

Servidor é papel de processo. O mesmo computador pode rodar cliente e servidor.

"IP e porta significam a mesma coisa."

O IP identifica a interface na rede; a porta distingue pontos de comunicação dentro do host.

"Escrever uma URL inicia o servidor."

Texto não executa nada. Sem processo em execução escutando na porta, não há quem atenda.

"TCP garante sucesso em qualquer situação."

TCP entrega de forma ordenada e confiável enquanto a comunicação é viável; se o caminho falha, a troca falha.

// demonstração

Criando um mini servidor com Python

Artefato de hoje: `redes-ii/aula01/laboratorio_rede.py` . Ele guarda e exibe as partes do endereço

// terminal · Windows

O interpretador precisa estar disponível antes da execução

PowerShell — redes-ii

```
PS> py --version
```

```
Python 3.11.x
```

```
// se py não for reconhecido:
```

```
PS> python --version
```

Nenhum comando responde? O Python não está instalado — isso é tarefa de preparação do laboratório, não da aula.

Uma pasta da aula mantém os arquivos organizados

terminal aberto no diretório da disciplina

```
PS> mkdir redes-ii
```

```
PS> cd redes-ii
```

```
PS> mkdir aula01
```

```
PS> cd aula01
```

NO VS CODE

1. Arquivo → Abrir Pasta → selecione `aula01`.

2. Crie o arquivo `laboratorio_rede.py`.

Abra pelo menu — não dependa do comando `code` no terminal.

```
redes-ii/ └─ aula01/ └─ laboratorio_rede.py
```

// código · laboratorio_rede.py (parte 1)

As variáveis guardam cada parte do endereço

```
# Endereço do serviço que usaremos na próxima aula.  
  
protocolo = "http"  
servidor = "127.0.0.1"  
porta = 8000  
recurso = "/"  
  
url =  
f"{protocolo}://{servidor}:{porta}{recurso}"
```

"texto" — entre aspas, é cadeia de caracteres.

8000 — sem aspas, é número.

f"..." — f-string: o que está entre chaves é substituído pelo valor da variável.

VALOR CALCULADO DE url

http://127.0.0.1:8000/

Ainda não é saída do terminal: este trecho não imprime.

// código · laboratorio_rede.py (completo)

O script completo exhibe os dados sem abrir conexão

```
# Endereço do serviço que usaremos na próxima aula.
protocolo = "http"
servidor = "127.0.0.1"
porta = 8000
recurso = "/"

url = f"{protocolo}://{servidor}:{porta}{recurso}"

print("Cliente: navegador")
print(f"Servidor:{servidor}")
print(f"Porta:{porta}")
print(f"URL preparada:{url}")
print("Nenhuma conexão foi realizada por este script.")
```

SAÍDA ESPERADA

```
Cliente: navegador
Servidor: 127.0.0.1
Porta: 8000
URL preparada: http://127.0.0.1:8000/
Nenhuma conexão foi realizada por este
script.
```

Uma URL impressa pelo Python não
comprova que exista um servidor atendendo
nela.

// terminal • execução

A saída confirma que o arquivo executou corretamente

PowerShell – redes-ii/aula01

```
PS> py laboratorio_rede.py
Cliente: navegador
Servidor: 127.0.0.1
Porta: 8000
URL preparada: http://127.0.0.1:8000/
Nenhuma conexão foi realizada por este script.
```

CRITÉRIOS DE CONFERÊNCIA

Endereço correto: 127.0.0.1

Porta 8000

URL completa com protocolo e caminho

Aviso de que nenhuma conexão ocorreu

Linux e macOS: python3 laboratorio_rede.py

// terminal • execução

Vamos agora iniciar um serviço HTTP

PowerShell – redes-ii/aula01

```
PS> py -m http.server 8000 --bind 127.0.0.1
```

Linux ou macOS – redes-ii/aula01

```
PS> python3 -m http.server 8000 --bind 127.0.0.1
```

Saída esperada

Serving HTTP on 127.0.0.1 port 8000...

// bloco D • prática • 25 min

Você vai reproduzir, modificar e conferir o script

1 min

Orientação

10 min

Reprodução do script

10 min

Modificação e
revisão em dupla

2 min

Erros operacionais

2 min

Conferência da
entrega

Seu arquivo deve produzir a saída de referência

- 1 Criar a pasta `redes-ii/aula01` e o arquivo.
- 2 Digitar o código apresentado.
- 3 Salvar e executar no terminal.
- 4 Comparar a saída com a referência.
- 5 Capturar a evidência como `evidencia_aula01.png`.

CRITÉRIO DE CONCLUSÃO

O arquivo executa sem erro e a URL `http://127.0.0.1:8000/` aparece na saída.

Travou? Confira a pasta do terminal, a extensão `.py`, se salvou o arquivo e o comando usado.

Uma mudança controlada altera uma parte da URL

- 1 Prever a saída ao trocar a porta para 9000.
- 2 Alterar somente a porta e executar.
- 3 Registrar previsão e resultado em `registro_aula01.txt`.
- 4 Pedir ao colega que explique o que mudou.
- 5 Restaurar a porta 8000 e executar novamente.

O REGISTRO DEVE RESPONDER

Quem será o cliente na próxima aula?

O que significa 127.0.0.1?

Qual parte da URL mudou?

A alteração iniciou algum serviço? Por quê?

Uma alteração por teste: é assim que se sabe a causa do resultado.

O erro indica qual parte da execução precisa de correção

`py : o termo 'py' não é reconhecido...`

O terminal não encontrou o interpretador.

→ Tente `python --version` ; se também falhar, o Python não está instalado.

`can't open file 'laboratorio_rede.py'`

Erro de localização: o arquivo não está onde o terminal está.

→ Confira a pasta atual e o nome do arquivo, inclusive a extensão.

`SyntaxError: unterminated string literal`

Aspas abertas e não fechadas: o Python não conseguiu ler o código.

→ Feche as aspas na linha indicada pela mensagem.

`NameError: name 'servidorr' is not defined`

Nome usado é diferente do nome criado.

→ Use exatamente o mesmo nome da variável, com a mesma grafia.

As mensagens variam por versão e sistema — leia o padrão. Nenhum desses é erro de rede: aqui não há comunicação.

// entrega da aula 01

A entrega demonstra execução e compreensão

[] Arquivo `laboratorio_rede.py` salvo.

[] Script executa sem erro.

[] Porta final restaurada para 8000.

[] URL final correta.

[] Registro com respostas e comparação.

[] Evidência da execução disponível.

RESULTADO OBSERVÁVEL

redes-ii/aula01/

└─ `laboratorio_rede.py`

└─ `registro_aula01.txt`

└─ `evidencia_aula01.png`

Cada informação tem um papel na comunicação

01 O cliente inicia a solicitação.

02 O servidor precisa estar executando.

03 O endereço IP e a porta ajudam a localizar o serviço.

04 O protocolo estabelece regras de comunicação.

05 O script desta aula prepara informações, sem acessar o serviço.

// desafio para casa

Você consegue prever o endereço antes de executar

- 1 Criar uma cópia chamada `desafio_aula01.py`.
- 2 Alterar o caminho para `/status`.
- 3 Prever a nova URL antes de executar.
- 4 Registrar se a alteração cria uma página ou um serviço.
- 5 Preservar o arquivo original para a próxima aula.

ENTREGA

Código e resposta curta no início da aula seguinte.

ESCREVA SUA PREVISÃO
`http://`

Na próxima aula, o endereço terá um servidor atendendo

O QUE FAREMOS

Executar um servidor HTTP local com Python

E acessá-lo pelo navegador em `http://127.0.0.1:8000/`. O endereço de hoje passa a ter alguém atendendo.

O QUE TRAZER

Pasta `redes-ii/aula01` completa.

Script original com porta 8000.

Python e navegador disponíveis na máquina.

// referências

A documentação ajuda a conferir conceitos e comandos

Kurose, J. F.; Ross, K. W. *Redes de Computadores e a Internet*, 8ª edição — capítulos 1 e 2

Camadas, aplicações e o modelo cliente-servidor.

Tutorial oficial do Python

docs.python.org/3/tutorial/

Documentação de `http.server`

docs.python.org/3/library/http.server.html

Documentação de `socket` — leitura futura, não é pré-requisito

docs.python.org/3/library/socket.html